

Comprehensive Project Report: AI Brand Voice Generator

Author: Niket Sah | B.Tech, Computer Science

Project Track: AI, ML & Generative AI

Executive Summary

The **AI Brand Voice Generator** is an intelligent, multi-format marketing content creation suite. Powered by the Google Gemini API, Streamlit, and a relational SQLite database, this application allows marketers, founders, and content creators to extract a brand's unique personality (tone, vocabulary, and emotional rhythm) from sample texts. It utilizes this extracted "voice DNA" to instantly generate highly targeted marketing assets across multiple formats while persisting all generation history in a structured database.

1. Brainstorming & Ideation

Problem Statement

Maintaining a consistent brand voice across various marketing channels (social media, email, blogs) is a time-consuming challenge for businesses. Manual content creation often leads to tonal inconsistencies, while generic AI tools produce robotic, off-brand text that lacks the company's unique emotional resonance.

Proposed Solution

An intuitive web application that analyzes raw sample copy provided by the user, dynamically constructs a system-level Brand Voice Prompt, and uses few-shot prompting techniques to generate ready-to-publish, on-brand content.

2. Requirement Analysis

Functional Requirements

- **Brand Voice Extraction:** The system must analyze raw text to determine tone, style, and vocabulary.
- **Multi-Format Generation:** The system must generate Social Media Posts, Marketing Emails, Taglines, Ad Headlines, and Blog Intros.
- **Iterative Refinement:** Users must be able to submit feedback to refine and edit the generated text dynamically.
- **Data Persistence:** All brand profiles, input samples, generation requests, and refined outputs must be saved to a database.
- **Clipboard Integration:** One-click functionality to copy output text.

Non-Functional Requirements

- **Performance:** The AI must generate content rapidly with fallback model handling.
- **Usability:** The UI must be clean, responsive, and visually structured using modern CSS.
- **Security:** API keys must be securely loaded via environment variables and hidden from the frontend.

3. Project Design Phase

System Architecture

The application follows a standard three-tier architecture:

1. **Presentation Layer:** A web interface built with Streamlit, utilizing custom HTML/CSS containers for a polished user experience.
2. **Application Logic Layer:** Python scripts handling API routing, prompt engineering, strict grounding rules, and error handling.
3. **Data Layer:** A localized SQLite3 database configured with strict foreign key constraints.

Entity-Relationship Diagram (ERD) Overview

The relational database is normalized into 6 interconnected tables:

1. **USER_ACCOUNT:** Stores user credentials and authentication data.
2. **BRAND_PROFILE:** Links to the user and stores industry, target audience, and extracted voice guidelines.
3. **SAMPLE_TEXT:** Stores the raw reference materials used to train the brand profile.
4. **CONTENT_REQUEST:** Logs the specific topic, format, and user instructions for a generation task.
5. **GENERATED_CONTENT:** Stores the AI's output and the specific model version used.
6. **CONTENT_REFINEMENT:** Logs any feedback provided by the user and the resulting updated text.

4. Project Planning Phase

Technology Stack Selection

- **AI Engine:** Google Gemini API (utilizing gemini-1.5-flash to gemini-3.6-flash architectures).
- **Frontend Framework:** Streamlit (Python).
- **Backend & Persistence:** Python 3.12+, SQLite3.
- **Libraries:** google-generativeai, python-dotenv (security), pyperclip (clipboard management).

Development Milestones

1. Design and initialize the 6-table SQLite database schema.
2. Develop the AI engine wrapper with few-shot templates and strict grounding rules.
3. Build the Streamlit UI (sidebar configuration, main content area).
4. Integrate the backend database with the frontend to enable stateful sessions.
5. Implement UI/UX enhancements (CSS styling, responsive layout bug fixes).

5. Project Development Phase

Core AI Logic Implementation

The AI engine (`ai_brand_voice_generator.py`) was developed with a two-step generation pipeline:

- **Voice Learning System:** A meta-prompt instructs Gemini to act as a Brand Strategist, extracting tone and vocabulary from user samples into a reusable system prompt.
- **Content Generation Engine:** Employs **Few-Shot Prompting**. Each content type (e.g., Email, Ad Headline) is paired with explicit format examples to ensure the AI understands the desired output structure.
- **Strict Grounding:** A global rule (CRITICAL GROUNDING RULE) is injected into every system instruction to strictly prohibit conversational filler (e.g., *"Here is your generated post:"*).

Database Integration

The `database.py` script was written to initialize the schema using `PRAGMA foreign_keys = ON`. Functions like `create_brand_profile` and `save_content_generation` were wrapped in `try-except` blocks within the UI to ensure seamless data persistence without interrupting the user experience.

User Interface Development

The frontend (`app.py`) utilizes Streamlit's `st.container(border=True)` and `st.columns()` to create a modern, card-based layout. Custom CSS was injected to render a gradient header and styled sub-headers.

6. Project Testing

AI Output Validation

- **Testing Fallback Mechanisms:** Verified that if a primary model endpoint fails, the system automatically routes the request to a fallback model.
- **Format Accuracy:** Tested output formatting to ensure Markdown lists, emojis, and hashtags rendered correctly without raw HTML leakages.

Database & UI Debugging

- **Schema Verification:** Resolved initial `NameError` execution bugs by properly defining table verification functions before execution.
- **UI Rendering Fixes:** Corrected Markdown indentation bugs that previously caused HTML layout blocks to render as raw code snippets.

7. Project Documentation

Comprehensive documentation was created to ensure the project is open-source friendly and maintainable:

- **README.md:** Contains the project overview, tech stack, key technical capabilities, and step-by-step quick start instructions.
- **.env:** Configured to securely handle the `GEMINI_API_KEY` while keeping credentials out of

version control.

- **requirements.txt:** Locks dependency versions (e.g., streamlit>=1.30.0, google-genai>=2.3.0) for reliable cloud deployments.
- **.gitignore:** Ensures `__pycache__`, virtual environments, and local `.db` files are excluded from the repository.

8. Project Demonstration

Deployment Setup

The application was successfully deployed to the cloud via **Streamlit Community Cloud**.

- **Continuous Integration:** Linked directly to the main branch of the GitHub repository, allowing for automated rebuilds upon new code commits.
- **Secure Execution:** The API keys were injected directly into the Streamlit Cloud Secrets manager, ensuring the live application functions securely without exposing sensitive credentials.

Final Workflow

1. The user inputs a Brand Name, Campaign Purpose, and raw Sample Text.
2. The application processes the text, extracts the brand DNA, and connects to the Gemini API.
3. The requested marketing asset is rendered in a styled container.
4. The user copies the text to their clipboard or utilizes the refinement loop to request stylistic adjustments.
5. All actions are silently and successfully logged in the backend SQLite database.